Arrays - Grundlagen

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Vergleich von Iteration und Rekursion
- Stack-Speicher

Was wird in diesem Foliensatz besprochen?

- Grundlagen von Arrays
- Eindimensionale Arrays
- Umgang mit Arrays

Was lernen Sie kennen?

- Eindimensionale Arrays in Java
- Zugriff auf Arrays

Motivation

Problem

- Wir möchten z. B. 1000 Messwerte abspeichern und bearbeiten
- 1000 Variablen?

Lösung

- Eine Struktur, die das Verwalten vieler Werte erleichtert

Array

Ein Array ist eine Zusammenfassung gleichartiger Elemente

Ein Array ist eine Datenstruktur

Datenstrukturen

- Legen größere Mengen an Daten strukturiert ab
- Unterstützen den gezielten Zugriff auf die Daten

Arrays deklarieren

Form: `Datentyp[] Name`
Beispiel: `int[] number;`

Legt nur fest, dass `number` auf ein Array von ganzen Zahlen (ein `int`-Array) zeigt
Es wird noch **keine Größe** angegeben!

Anlegen von Arrays

Anlegen bei Deklaration
`int[] arr = new int[10];`

Späteres Anlegen

`int[] arr;`
…
`arr = new int[10];`

new-Operator

Bei Aufruf des new-Operators wird Speicher angefordert (abhängig von der Größe)

Es wird ein Array-Objekt erzeugt

Größe des Arrays

- Ist ein ganzzahliger Ausdruck (kann auch berechnet werden)

Arrays – Initialisierung

Alle Array-Elemente (Array-Einträge) werden automatisch initialisiert

- 0 bei int (auch bei byte und short)
- 0L bei long
- 0.0f bei float
- 0.0d bei double
- '\u0000' bei char
- false bei boolean
- null bei String (und anderen Referenztypen)

Anlegen mit eigener Initialisierung

```
int[] arr = new int[]{3, 4, 5, 6, 7};
// oder
int[] arr;
arr = new int[] {3, 4, 5, 6, 7};
// oder
int[] arr = {3, 4, 5, 6, 7};
```

Array hat hier immer die Länge 5
Array arr enthält hier immer die Elemente 3, 4, 5, 6, 7

Arrays – Index

- Jedem Element eines Arrays ist ein Index vom Typ int zugewiesen
- Indexzählung beginnt bei **0**
- Indexzählung geht bis **(Länge des Arrays) - 1**

Beispiel für Index (schematisch)

```
int[] arr = new int[10];
```

Index	0	1	2	3	4	5	6	7	8	9
Inhalt	0	0	0	0	0	0	0	0	0	0

Größe des Arrays
• Kann mit arr.length abgefragt werden (z. B. 10 im obigen Beispiel)

Zugriff auf einzelne Elemente

Zugriff
• Indexwert in eckigen Klammern (z. B. array[3]) • Kann wie eine Variable verwendet werden

```java
int[] array = new int[5];
array[0] = 2;
array[1] = array[0] + 3;
for (int i = 0; i < array.length; i++) {
    System.out.println(array[i]);
}
```

```
2
5
0
0
0
```

Ausdruck als Array-Index

```java
int[] array = new int[5];
int n = array.length;
array[0] = 2;
array[1] = array[0] + 3;
array[2]++;
array[3] = array[n - 3];
array[4] = array[array[2]];
for (int i = 0; i < n; i++) {
    System.out.println(array[i]);
}
```

```
2
5
1
1
5
```

Der Typ des Indexausdrucks muss `int` (bzw. `short`, `byte` oder `char`) sein

Indexfehler

Indexwert muss gültig (im Bereich des Arrays) sein

Sonst **Laufzeitfehler**, d. h. das Programm wird während der Ausführung sofort unterbrochen

Programm wirft eine **ArrayIndexOutOfBoundsException**

Laufzeit ≠ Übersetzungszeit
Ausnahmen (Exceptions) werden zur Laufzeit geworfen

Indexfehler – Beispiel

```java
int[] array = new int[5];
array[5] = 1;
```

Ausgabe in IntelliJ bei Ausführung

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
                    at Tester.main …
```

Beispiel (Operationen auf Arrays)

```
public class ArrayTest {

    public static void main(String[] args) {
        int n = 5;
        double[] data = new double[n];
        double sum = 0.0;
        for (int i = 0; i < n; i++) {
            data[i] = Math.random() * 100;
        }
        for (int i = 0; i < n; i++) {
            sum += data[i];
        }
        double average = sum/n;
        for (int i = 0; i < n; i++) {
            System.out.println("Element at position " + i  + ": " + data[i]);
        }
        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }

}
```

```
Element at position 0: 71.13847532247067
Element at position 1: 40.372869246156554
Element at position 2: 68.0432110287555
Element at position 3: 9.694335299857093
Element at position 4: 45.36966934719746
Sum: 234.61856024443728
Average: 46.92371204888745
```

„Initialisieren“ des Arrays mit zufälligen Werten

Beispiel (Inhalt eines Arrays umdrehen)

```java
public class ArrayTest2 {

    public static void main(String[] args) {
        int n = 10;
        int[] data = new int[n];
        for (int i = 0; i < n; i++) {
            data[i] = (int) (Math.random() * 100);
        }
        for (int i = 0; i < n; i++) {
            System.out.print(data[i] + "\t");
        }
        System.out.println();
        for (int i = 0; i < n/2; i++) {
            int temp = data[i];
            data[i] = data[n-i-1];
            data[n-i-1] = temp;
        }
        for (int i = 0; i < n; i++) {
            System.out.print(data[i] + "\t");
        }
    }
}
```

```
26	37	28	54	72	50	64	42	31	8
8	31	42	64	50	72	54	28	37	26
```

Beispiel (Bestimmen aller Primzahlen bis zu einer vorgegebenen Zahl)

Alle Zahlen 2, 3, 4, … bis zu einem Maximalwert n werden betrachtet

Die kleinste unmarkierte Zahl ist immer eine Primzahl

Wurde eine Primzahl gefunden, werden alle Vielfachen dieser Primzahl als zusammengesetzt markiert

Es wird die nächstgrößere nicht markierte Zahl bestimmt

- Es werden wieder alle Vielfachen markiert und das Verfahren fortgeführt

Ablauf (Beispiel)

Ausgangslage

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Alle Vielfachen von 2 markieren

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Alle Vielfachen von 3 markieren

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Alle Vielfachen von 5 markieren

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

Code in TUWEL – Sieve.java

Iterator-Form der for-Schleife

Ausgabe mit herkömmlicher for-Schleife

```
int[] arr = {3, 4, 5, 6, 7};
for (int a = 0; a < arr.length; a++)
    System.out.println(arr[a]);
```

Mit der Iterator-Form (foreach-Schleife)

```
int[] arr = {3, 4, 5, 6, 7};
for (int i : arr)
    System.out.println(i);
```

- Schleifenrumpf wird für jedes Element des Arrays ausgeführt
- Elementvariable (im Beispiel i)
  - Erhält den Wert des jeweils nächsten Array-Elements
  - Kann verändert werden, der Array-Inhalt bleibt aber gleich
- Nur kompletter Durchlauf (aufsteigend)

char-Arrays und Strings

```
char[] c = {'H', 'e', 'l', 'l', 'o'};
String s = "Hello";
```

Einige Unterschiede
• Ein String ist **unveränderlich**, ein char-Array nicht • Strings können mit + verkettet werden, char-Arrays nicht • String kann einfacher erzeugt werden (siehe obiges Beispiel)

Array von Strings

```
String[] ts = new String[]{ "Hello", "World", "Bye" };
for (String t : ts) {
    System.out.println(t);
}
```

```
Hello
World
Bye
```

```
String[] ts = new String[3];
for (String t : ts) {
    System.out.println(t);
}
```

```
null
null
null
```

Standardinitialisierung

Array von Strings – null-Wert

null-Wert kann ausgegeben werden

null-Wert kann nicht bei einem Aufruf einer Methode verwendet werden!

- Ausnahme (**NullPointerException**) wird geworfen

```
String[] ts = new String[3];
for (String t : ts) {
    System.out.println(t.length());
}
```

Erzeugt Ausnahme!
Methodenaufruf erfordert einen gültigen String!

Array von Strings – Überprüfungen

```
for (String t : ts) {
    if (t != null) {
        System.out.println(t.length());
    }
}
```

Sicherstellen, dass Elemente Strings sind
Wenn Array möglicherweise nicht mit Strings belegt ist auf null überprüfen!